Typed programming languages are understood via their type theories. It's hard to pin down a precise definition of a type theory in general. For our purposes, we can understand a type theory in a naturally deductive style as a having type formation rules, term introduction rules, term elimination rules, and β/η rules constraining the behavior of terms

For example, for product types

$$\frac{\Gamma \vdash A \text{ Type} \quad \Gamma \vdash B \text{ Type}}{\Gamma \vdash A \times B \text{ Type}} \times\text{Formation}$$

$$\frac{\Gamma \vdash e : A \times B}{\Gamma \vdash \pi_1 e : A} \times E_1 \qquad \frac{\Gamma \vdash e : A \times B}{\Gamma \vdash \pi_2 e : B} \times E_2$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash (a, b) : A \times B} \times I \qquad \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \pi_1 (a, b) \equiv a : A} \times \beta_1$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \pi_2 (a, b) \equiv b : B} \times \beta_2 \qquad \frac{\Gamma \vdash e : A \times B}{\Gamma \vdash (\pi_1 e, \pi_2 e) \equiv e : A \times B} \times \eta$$

hese rules specify the behavior of the roduct type fully. Via the η-rule, $e : A \times B$ is entirely determined by its projections. We may likewise give rules characterizing other connectives like the sum types, function types and so on.

Programming languages comprise terms inductively generated via these introduction and elimination rules, and its important to ecognize that they are not simply ad-hoc inductive definitions. Instead, they are described very principledly by giving rules for introducing and eliminating each typing connective.

In fact, we can abstract over these otions of intro/elim/β/η using the language of category theory.
This is an old and well-established idea going back to at least the 60s ith the work of Lawvere, and perhaps ven further. In summary, we can nderstand programming languages as a generalized form of algebra, and in this manner we can give precise algebraic definition of a model. That is, each typing connective lefines a universal categorical structure, and the correct notion of model for theory is a category interpreting all the requisite universal structures

t's make this concrete for a simple language with product types, as above.

ategorically, the analog of a product type is a product object. We say that P is a product of A and B if
- There are morphisms $\pi_1 : C[P, A]$
  $\pi_2 : C[P, B]$
- Such that for all Q w/ $f : C[Q, A]$
  $g : C[Q, B]$

there exists a unique morphism $\varphi : C[Q, P]$ such that
$\varphi * \pi_1 \equiv f \qquad \varphi * \pi_2 \equiv g$
write φ as (f, g)

How does this relate back to type theory?
e can view (simple) type theory through the lens of category theory by interpreting a term

$\Gamma \vdash e : A$ as a morphism

$e : C[\Gamma, A]$

In this manner, we recover the intro/elim/β/η precisely through the notion of categorical product.
intro is the morphism (f, g)
elims are the projection morphisms π_1, π_2
β rules are the equalities $(f, g) * \pi_1 \equiv f$
$(f, g) * \pi_2 \equiv g$

and η is the captured via uniqueness of (f, g) between Q and P

If we wish to understand other type structures, we simply provide corresponding universal properties.
For instance, sum types

sum types are universal types w/ coprojections

$$\begin{tikzcd} & C & \\ & A+B \arrow[u, dashed, "{\exists! [f,g]}"] & \\ A \arrow[ur, "\sigma_1"] \arrow[uur, bend left, "f"] & & B \arrow[ul, "\sigma_2"'] \arrow[uul, bend right, "g"'] \end{tikzcd}$$

↳ categorical coproduct object

or function types

function types are universal types supporting an application operation

$$\begin{tikzcd} A \Rightarrow B \times A \arrow[r, "app"] & B \\ C \times A \arrow[u, "\lambda f \times id"] \arrow[ur, "f"'] & \end{tikzcd}$$

categorical exponential object

In this manner, each type structure can be modularly captured by the specification of some universal structure in a category

We may build a category $\mathcal{L}_T$ that captures precisely the syntax of a type theory T

As objects, it has contexts of the type theory
and as morphisms it has substitutions between contexts

↳ Or perhaps better phrased directly as terms into types representing contexts

Moreover, $\mathcal{L}_T$ is unique among all models of T in that there is a functor from $\mathcal{L}_T$ into any category C describing a model of T.

For instance, for a theory T describing a language w/ products over a signature Σ of generating data

↳ Reconcile Σ v. T as signature

$\mathcal{L}_\times[\Sigma]$ we write $CC[\Sigma]$ to describe the free cartesian category over Σ.
↳ just means that all products exist (binary and terminal)

Given any other cartesian category C with an interpretation $\iota : \Sigma \rightsquigarrow C$, we may define a functor $rec_\iota : CC[\Sigma] \longrightarrow C$ such that
rec_ι strictly preserves the Cartesian structure and strictly preserves the interpretations of Σ

That is, we map a syntactic description of a product type to the semantic interpretation of a product type and we send generators of Σ to their interpretation under ι.

↳ This is all pretty standard but I need to give enough detail to get the "syntax is free categories" idea across.

rec_ι is analogous to the definition of the recursor for an inductive datae type in type theory
For instance lists over a type A have a recursor

$rec_{List} : \forall \{B\} \to B \to (A \to B \to B)$
$\to List\ A \to B$
$rec_{List}\ b\ \_\ [\,] = b$
$rec_{List}\ b\ f\ (x :: xs) = f\ x\ (rec_{List}\ b\ f\ xs)$

b gives an interpretation of the empty list and f gives an interpretation of cons.
rec is designed precisely so that it maps [] to b and cons to action by f — i.e. so it preserves the interpretations

However, there is a stronger freeness property witnessed by types in dependent type theory; they have a dependent elimination principle where the return type (or dependent motive) may be a dependent type

$elim_{List} : \forall \{B : List\ A \to Type\} \to$
$B\ [\,] \to$
$(\forall \{a\ l\} \to B\ l \to B(a :: l)) \to$
$(l : List\ A) \to B\ l$
$elim_{List}\ b\ \_\ [\,] = b$
$elim_{List}\ b\ f\ (x :: xs) = f\ (elim_{List}\ b\ f\ xs)$

$elim_{List}$ is nearly the same code as rec_{List} but is more powerful due to its dependent nature

↳ You could actually implement $elim_{List}$ using rec_{List} in Cubical Agda

by letting the return type of the recursor be $\sum\limits_{as : List\ A} B\ as$

let $e = rec\ \{B = \sum\limits_{as} B\ as\}\ ([\,], b)\ (\lambda\ a\ (ys, b_{ys}) \to (a :: ys, f\ b_{ys}))$
xs

↳ has type $\sum\limits_{as} B\ as$ however when you inspect e.snd you care over the wrong index. Instead $e.snd : B\,(e.fst)$ and we have propositionally that $e.fst = xs$, so we could transport over that proof of equality
↳ To avoid the transport, we can just use the eliminator directly as it definitionally over the correct list

Just as with lists, the free cartesian category has a stronger freeness property than just the functor into any other cartesian category

What is the categorical analogue of a dependent type? How do we phrase a dependent eliminator via category theory?

↳ Need to at this point have made a stronger case for why we want to use category theory in the first place

The appropriate categorical notion of dependent type for us is provide by displayed categories

↳ Introduce them
↳ Motivate that they are a repackaging of existing ideas in a manner favorable to the proof assistant
↳ Just as $elim_{List}$ lets us avoid mentioning equality of lists propositionally, displayed categories give a degree of strictification that avoids the propositional equality of objects in a category
↳ Moreover, displayed morphisms also elide equality of morphisms in the base category
↳ Equality of objects is evil

The dependent eliminator for the free cartesian category CC[Σ] is captured by a global section of any displayed category D over CC[Σ] that

- has an interpretation of generating data $\iota : \Sigma \rightsquigarrow D$
- is an appropriate notion of cartesian
  ↳ We define displayed cartesian categories later in the paper in more detail

We often visualize these sections as

$$\begin{tikzcd} D \arrow[d] \\ CC[\Sigma] \arrow[u, dotted, bend right, "elim_\iota"'] \end{tikzcd}$$